

캡스톤 디자인 06분반

Real-Time Object Detection Based Smart Safety System

2022652 서주호

1922819 정세훈

2121944 김은영

2221301 정혁수

1823933 백시우

CONTENTS

01	02	03	04	05	06
프로젝트 선정 이유 - 사회적 문제 - 기대효과 및 활용방안	개발 환경 및 주요 기능 - 프로젝트 목표 - 프로젝트 개발 환경 - 주요 기능	프로젝트 수행내용 - 객체 탐지 시스템 - 현장 알림 시스템 - 웹 알림 시스템 - 앱 알림 시스템	결과물 - 프로젝트 작동	프로젝트 동작 시연 - 작동 영상	현실적 제한요소 - 제한요소

01

프로젝트 선정이유
사회적 문제



01

프로젝트 선정이유

사회적 문제

3배

안전대 미착용 시 사망률 증가

30%

장비 점검 시 사고 감소율

1280건

2024년 안전대 미착용 적발 건수

01 프로젝트 선정이유 기대효과 및 활용방안

2

효율적 관리 및 유지보수

- 관리인력의 피로도 감소
- 자동화 시스템의 유지보수 비용 감소

1

안전사고 예방 및 신속한 초기 대응

- 실시간 알림 제공
- 각종 센서를 통해 위험 상황을 인지함

3

실시간 데이터 수집 및 분석

- 카메라를 통해 얻는 데이터를 실시간으로 수집
- 데이터 분석을 통해 다양한 분야에 이용 가능

4

다양한 분야로의 확장성

- 공공장소 범죄, 학교 안전관리, 병원 환자관리
- 군에서의 활용

5

중대재해처벌법 대응

- 안전관리자를 대신할 수 있음

기대효과 및 활용방안



02

개발환경 및 주요기능
프로젝트 목표

Real-Time Object Detection Based Smart Safety System

STEP.01

STEP.02

STEP.03

STEP.04

CCTV 기반 객체인식

카메라, CCTV를 이용
AI기반 객체인식



안전 또는 위험 판단

각각의 상황에 따른
위험상태 판단



현장 알림

부저 및 LED를 통해 현장알림



웹, 앱 알림

서버를 통해
실시간으로 웹, 앱 알림 제공

02 개발환경 및 주요기능

프로젝트 목표



Real-Time Object Detection

HELMET ON/OFF

VEST ON/OFF

HELMET **ON**
VEST **ON**

Helmet LED, Alert
Vest LED, Alert
Status LED, Alert



SAFE

HELMET **OFF**
VEST **ON**

Helmet LED, Alert
Vest LED, Alert
Status LED, Alert



WARNING

HELMET **ON**
VEST **OFF**

Helmet LED, Alert
Vest LED, Alert
Status LED, Alert



WARNING

HELMET **OFF**
VEST **OFF**

Helmet LED, Alert
Vest LED, Alert
Status LED, Alert



DANGER

02

개발환경 및 주요기능

프로젝트 개발 환경



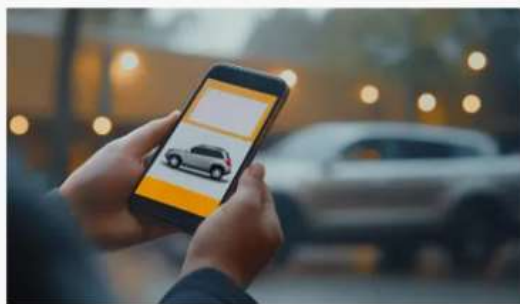
01.

Raspberry Pi 5



02.

Raspberry
Picamera V3



03.

AI Inference
Environment

YOLOv8



04.

개발 언어 및 도구

Python 3.11

OpenCV

Gpiozero

PyYAML

Ultralytics YOLO Framework

MIT App Inventor

02

개발환경 및 주요기능
주요 기능

Helmet



Vest



On-site Alert



WEB Alert



APP Alert



Dashboard



03 프로젝트 수행내용

객체 탐지 시스템

```
camera:
  width: 1280
  height: 720
  fps: 30
gpio:
  led_pin: 17
  buzzer_pin: 27
  buzzer_on_ms: 500
  buzzer_off_ms: 500

Inference:
  hello_hef_path: ""
  cpu_model_weight: "/home/eyes/Capstone/Smart-Safety/models/best.pt"
  cpu_model_weight_extra: "/home/eyes/Capstone/Smart-Safety/models/best2.pt"

  conf_thres: 0.25
  iou_thres: 0.45

logic:
  temporal_window: 12
  min_person_size_px: 10
  head_ratio: 0.30
  helmet_head_iou: 0.10
  helmet_min_conf: 0.35
  vest_torso_top_ratio: 0.18
  vest_torso_bottom_ratio: 0.98
  vest_torso_iou: 0.02
  vest_draw_top_ratio: 0.25
  vest_draw_bottom_ratio: 0.75

  vest_min_frames: 2
  vest_min_frames: 2

  alert_threshold: 0.50
  min_ppe_conf: 0.05
  draw_visual: true
  show_window: true
```

03 프로젝트 수행내용

객체 탐지 시스템, 규칙판정 - 1

```
persons = []
if self.person_id is not None:
    for d in dets:
        if d.get("cls") != self.person_id:
            continue

        box = d.get("box")
        if not box or len(box) != 4:
            continue

        px1, py1, px2, py2 = map(int, box)
        if px2 <= px1 or py2 <= py1:
            continue

        # 사람 박스 면적이 너무 작은 경우(멀리 있는 미세 detection) 제거
        area = (px2 - px1) * (py2 - py1)
        if self.min_px > 0 and area < self.min_px:
            continue # 너무 작은 사람 박스는 무시

        ph = py2 - py1
        if ph <= 0:
            continue
```

```
for d in dets:
    cls_id = d.get("cls")
    box = d.get("box")
    conf = float(d.get("conf", 1.0))

    if not box or len(box) != 4:
        continue

    # PPE 관련 클래스(helmet/no-helmet/vest)는 최소 confidence 체크
    if cls_id in (self.helmet_id, self.no_helmet_id, self.vest_id):
        if conf < self.min_ppe_conf:
            continue

    x1, y1, x2, y2 = map(int, box)
    if x2 <= x1 or y2 <= y1:
        continue

    w = x2 - x1
    h = y2 - y1
    area = w * h
    if area <= 0:
        continue
```

03 프로젝트 수행내용

객체 탐지 시스템 규칙판정 - 2

```
elif cls_id == self.vest_id:

    # vest도 너무 작은 경우는 person과 마찬가지로 제거
    if self.min_px <= 0 or area >= self.min_px:
        vest_boxes.append((x1, y1, x2, y2))

    color = (0, 255, 255) # 노랑
    label = "VEST"

else:
    # 기타 PPE 클래스(barefoot, shoes, mask, gloves 등)는
    # 현재 로직에서는 사용하지 않음 (헬멧/조끼에만 집중)
    continue
```

```
if cls_id in (self.helmet_id, self.no_helmet_id):

    # person이 하나라도 있는 경우, 머리(head) 영역과의 IoU로 필터링
    if persons:
        in_head_region = False
        for (px1, py1, px2, py2) in persons:
            # 사람 박스에서 머리 영역만 잘라내기
            hx1, hy1, hx2, hy2 = head_region( person_box: (px1, py1, px2, py2), self.head_r)

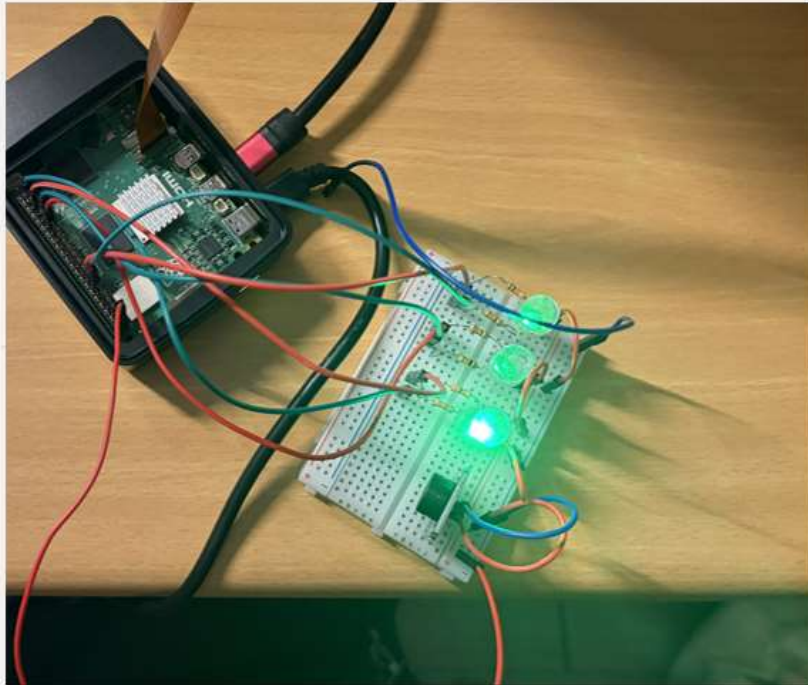
            # helmet/no-helmet 박스가 머리 영역과 충분히 겹치면 통과
            if iou( a: (x1, y1, x2, y2), b: (hx1, hy1, hx2, hy2)) >= self.hh_iou:
                in_head_region = True
                break

        if not in_head_region:
            # 사람 머리 영역과 거의 안 겹치면 오탐으로 보고 버림
            continue

    # 최종 카운트 및 색/라벨 설정
    if cls_id == self.no_helmet_id:
        no_helmet_cnt += 1
        color = (0, 0, 255) # 빨강
        label = "NO-HELMET"
    else:
        helmet_cnt += 1
        color = (0, 255, 0) # 초록
        label = "HELMET"
```

03 프로젝트 수행내용

현장 알림



```
from picamera2 import Picamera2
import time
import RPi.GPIO as GPIO

class Camera:
    def __init__(self, cam_cfg):
        self.picam2 = Picamera2()
        cfg = self.picam2.create_preview_configuration(
            main={"size": (cam_cfg["width"], cam_cfg["height"]), "format": "RGB888"},
            buffer_count=4
        )
        self.picam2.configure(cfg); self.picam2.start(); time.sleep(0.3)
    def read(self):
        import cv2
        frm = self.picam2.capture_array()
        return cv2.cvtColor(frm, cv2.COLOR_RGB2BGR)
    def close(self):
        self.picam2.stop()

class GPIOBoard:
    def __init__(self, cfg):
        GPIO.setmode(GPIO.BCM)
        self.leds = {}
        self.buzzer = None

        for key, pinset in cfg.items():
            if key in ["helmet_led", "vest_led", "status_led"]:
                r = pinset["r"]
                g = pinset["g"]
                b = pinset["b"]

                GPIO.setup(r, GPIO.OUT)
                GPIO.setup(g, GPIO.OUT)
                GPIO.setup(b, GPIO.OUT)

                GPIO.output(r, GPIO.HIGH)
                GPIO.output(g, GPIO.HIGH)
                GPIO.output(b, GPIO.HIGH)

                self.leds[key] = {"r": r, "g": g, "b": b}

            # (optional)
            if key == "buzzer_pin":
                GPIO.setup(pinset, GPIO.OUT)
                GPIO.output(pinset, GPIO.HIGH)
                self.buzzer = pinset

# -----
# Common Anode LED ON/OFF
# r, g
# 1 ON(LOW)
# 0 OFF(HIGH)
# -----
def set_rgb(self, name, r, g):
    led = self.leds[name]
    GPIO.output(led["r"], GPIO.LOW if r == 1 else GPIO.HIGH)
    GPIO.output(led["g"], GPIO.LOW if g == 1 else GPIO.HIGH)

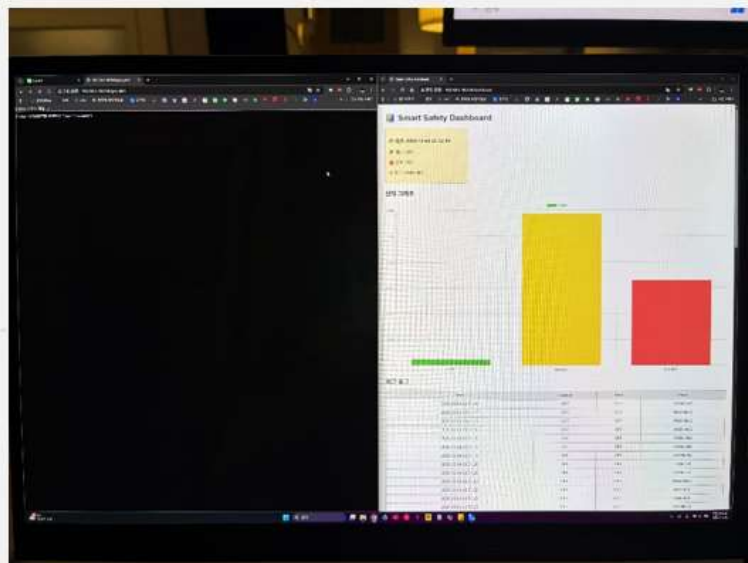
    GPIO.output(led["b"], GPIO.HIGH)

def buzz_on(self):
    if self.buzzer:
        GPIO.output(self.buzzer, GPIO.LOW)

def buzz_off(self):
    if self.buzzer:
        GPIO.output(self.buzzer, GPIO.HIGH)
```


03 프로젝트 수행내용

웹 알림



```
# =====
# 2) 라즈베리파이 → 서버 알림 전달 API
# =====
<@alert
@app.route("/alert", methods=["POST"])
def alert():
    global latest_alert
    data = request.get_json()
    alert_type = data.get("type", "unknown")

    latest_alert = {
        "type": alert_type,
        "time": time.time()
    }
    print("새 알림 수신:", latest_alert)
    return "ok"
```

```
def send_alert(alert_type): 1 usage
    """Flask 서버로 상태 전송"""
    try:
        r = requests.post(ALERT_URL, json={"type": alert_type}, timeout=1)
        print("[ALERT] Sent:", alert_type, "Status:", r.status_code)
    except Exception as e:
        print("[ALERT] Failed:", e)
```

```
def read_csv(): 1개의 사용 위치
    result = []
    try:
        with open("safety_log.csv", "r", encoding="utf-8") as f:
            for row in csv.DictReader(f):
                result.append(row)
    except FileNotFoundError:
        pass
    return result
```

```
<h1> Smart Safety Dashboard</h1>

<!-- 상태 카드 -->
<div id="status_card" class="card">
    <h3>⌚ 시간: <span id="latest_time"></span></h3>
    <p>🧯 헬멧: <span id="latest_helmet"></span></p>
    <p>🛡️ 조끼: <span id="latest_vest"></span></p>
    <p>▲ 상태: <span id="latest_final"></span></p>
</div>

<!-- 그래프 -->
<h2>상태 그래프</h2>
<canvas id="chart" width="400" height="200"></canvas>

<!-- 로그 테이블 -->
<h2>최근 로그</h2>
<table>
    <tr>
        <th>Time</th>
        <th>Helmet</th>
        <th>Vest</th>
        <th>Final</th>
    </tr>
    <tbody id="log_table">
    </tbody>
</table>
```

```
<script>

let chart = null;

function updateDashboard() {
    fetch("/dashboard_data")
        .then(r => r.json())
        .then(data => {

            // -----
            // 최신 상태 카드 업데이트
            // -----
            document.getElementById("latest_time").innerText = data.latest.time;
            document.getElementById("latest_helmet").innerText = data.latest.helmet;
            document.getElementById("latest_vest").innerText = data.latest.vest;
            document.getElementById("latest_final").innerText = data.latest.final;

            const card = document.getElementById("status_card");
            card.className = "card " +
                (data.latest.final === "SAFE" ? "safe"
                : data.latest.final === "WARNING" ? "warning"
                : "danger");

            // -----
            // 그래프 업데이트
            // -----
```


The screenshot shows an Android application running on a virtual device. The app has a blue header bar with the title "Screen1". Below the header, there is a list of items: "상태: 대기 중", "상태: 대기 중", "상태: 대기 중", and "시간: ". Below this list, there are several buttons: "수동 재로그인", "Dashboard", "CSV 로그 보기", "Log 보기", and "Clear_Log". To the right of the app interface, the Java code for the app is displayed. The code defines a class named "Screen1" which implements the "Runnable" interface. It contains a "run" method that calls "startScreen" and "stopScreen" methods. The "startScreen" method calls "startScreen" and "stopScreen" methods. The "stopScreen" method calls "stopScreen" and "startScreen" methods. The code also includes a "startScreen" method that calls "startScreen" and "stopScreen" methods. The code is written in Java and uses the "Runnable" interface.



```

do
  set global logs to call TinyDB1 -> GetValue
  tag -> logs
  valueIfTagNotThere -> create empty list
  set global newRecord to -> create empty dictionary
  set value for key -> time ->
  in dictionary -> get global newRecord ->
  to -> Label Time -> Text ->
  set value for key -> helmet ->
  in dictionary -> get global newRecord ->
  to -> Helmet Status -> Text ->
  set value for key -> vest ->
  in dictionary -> get global newRecord ->
  to -> Vest Status -> Text ->
  set value for key -> final ->
  in dictionary -> get global newRecord ->
  to -> Final Status -> Text ->
  add items to list -> list -> get global logs ->
  item -> get global newRecord ->
  call TinyDB1 -> StoreValue
  tag -> logs
  valueToStore -> get global logs

```

```

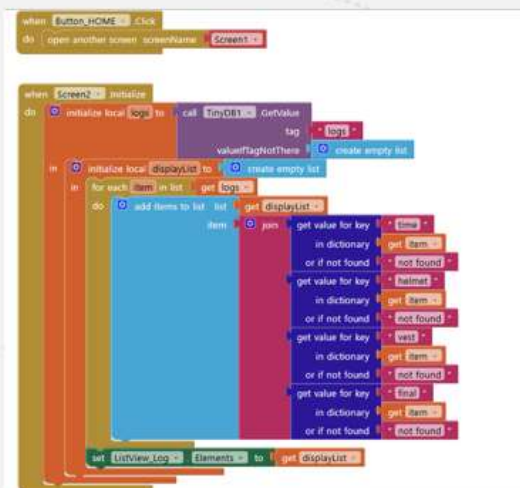
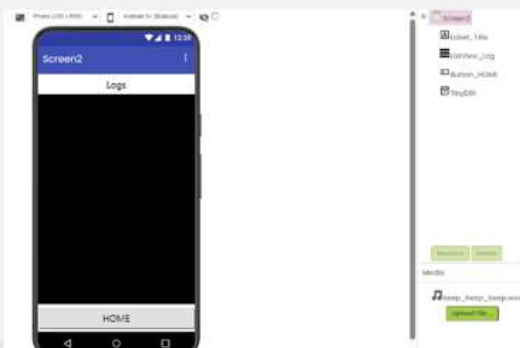
when Weight1 >= 0.001
  call responseCode responseType responseContent
  do get global alertDict to call Web3.js joinTextDecodeWithOctonaries joinText get responseContent
  set global alertType to get value for key type in dictionary or if not found get global alertDict
  set Label Time to Text to join get value for key time in dictionary or if not found get global alertDict
  if get global alertType == 0
    set Helmet_Status to Text to Helmet: ON
    set Vest_Status to Text to Vest: ON
    set Final_Status to Text to SAFE
    set Helmet_Status to TextColor to
    set Vest_Status to TextColor to
    set Final_Status to TextColor to
  else if get global alertType == 1
    set Helmet_Status to Text to Helmet: OFF
    set Vest_Status to Text to Vest: ON
    set Final_Status to Text to WARNING
    set Helmet_Status to TextColor to
    set Vest_Status to TextColor to
    set Final_Status to TextColor to
  else if get global alertType == 2
    set Helmet_Status to Text to Helmet: ON
    set Vest_Status to Text to Vest: OFF
    set Final_Status to Text to WARNING
    set Helmet_Status to TextColor to
    set Vest_Status to TextColor to
    set Final_Status to TextColor to
  else
    set Helmet_Status to Text to Helmet: OFF
    set Vest_Status to Text to Vest: OFF
    set Final_Status to Text to DANGER
    set Helmet_Status to TextColor to
    set Vest_Status to TextColor to
    set Final_Status to TextColor to
  if get global alertAlert == get global alertType
  then
    if get global alertType == 0
    then
      call Notify! ShowAlert notice [헬멧 착용을 권장합니다]
      call Sound! Play
    if get global alertType == 1
    then
      call Notify! ShowAlert notice [헬멧 미착을 주의합니다]
      call Sound! Play
    if get global alertType == 2
    then
      call Notify! ShowAlert notice [헬멧 착용을 권장합니다]
      call Sound! Play
    if get global alertType == 3
    then
      call Notify! ShowAlert notice [헬멧 미착을 주의합니다]
      call Sound! Stop
  call CopyData
  type
  time
  set global alertAlert to get global alertType

```

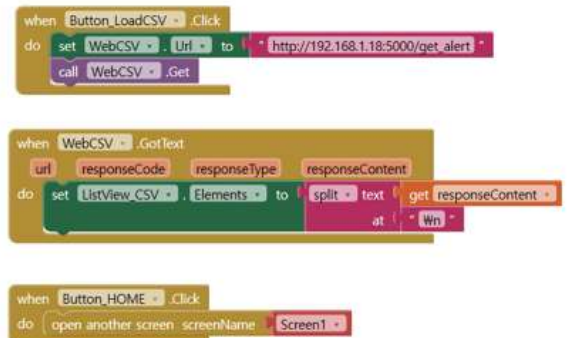
03 프로젝트 수행내용

앱 알림

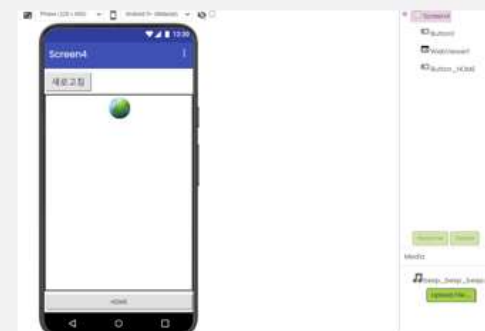
로그 조회



CSV 텍스트 로그 조회



웹 대시보드 연동



04 결과물

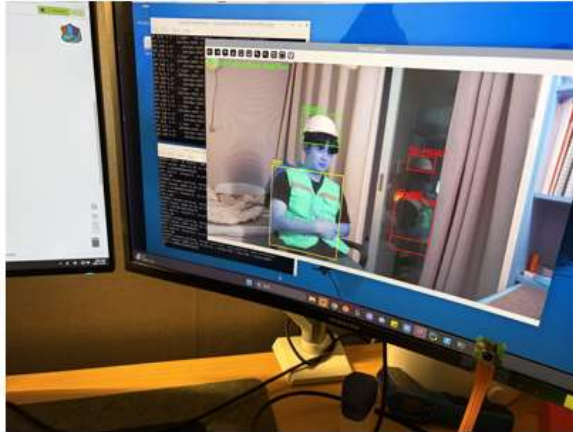
프로젝트 작동

01.

HELMET **ON**
VEST **ON**

SAFE

HELMET LED ●
VEST LED ●
STATUS LED ●

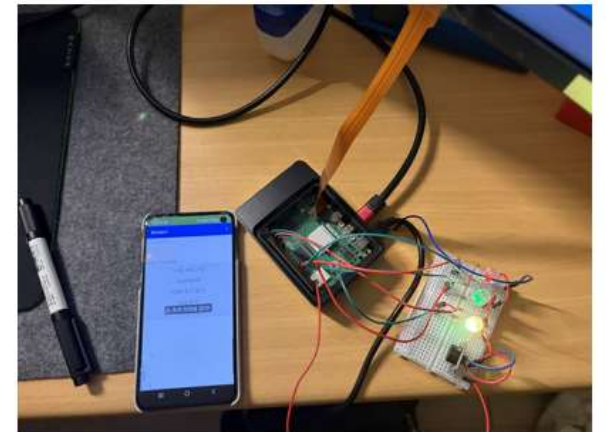
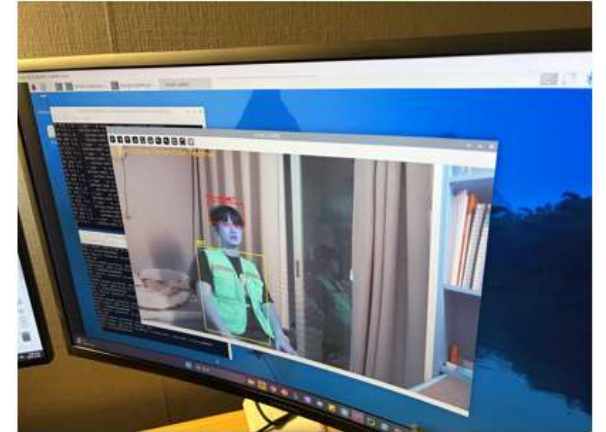


02.

HELMET **OFF**
VEST **ON**

WARNING

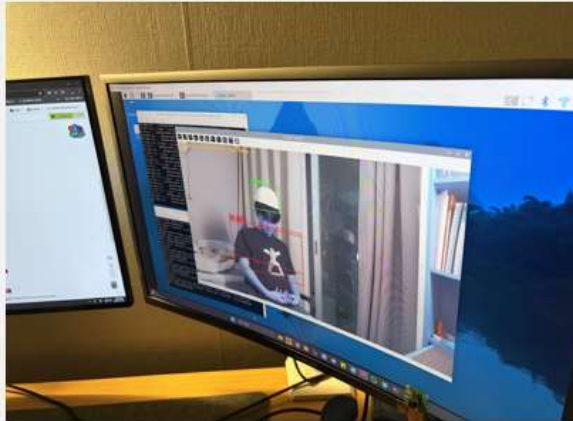
HELMET LED ●
VEST LED ●
STATUS LED ●



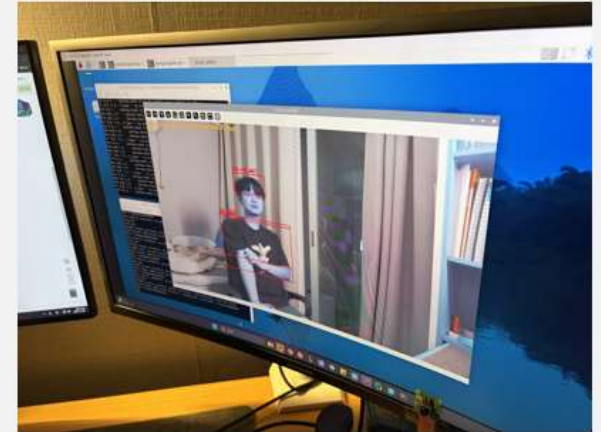
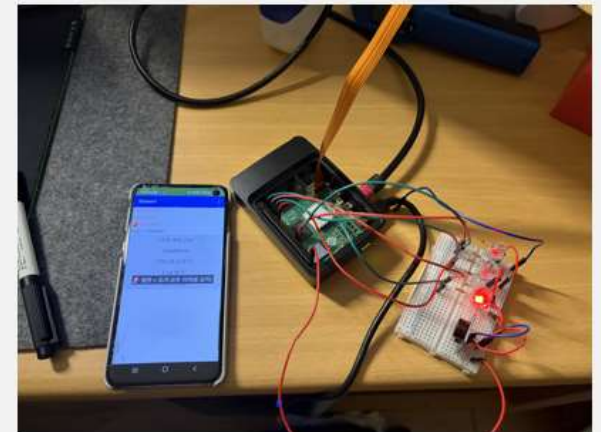
04 결과물

프로젝트 작동

03.

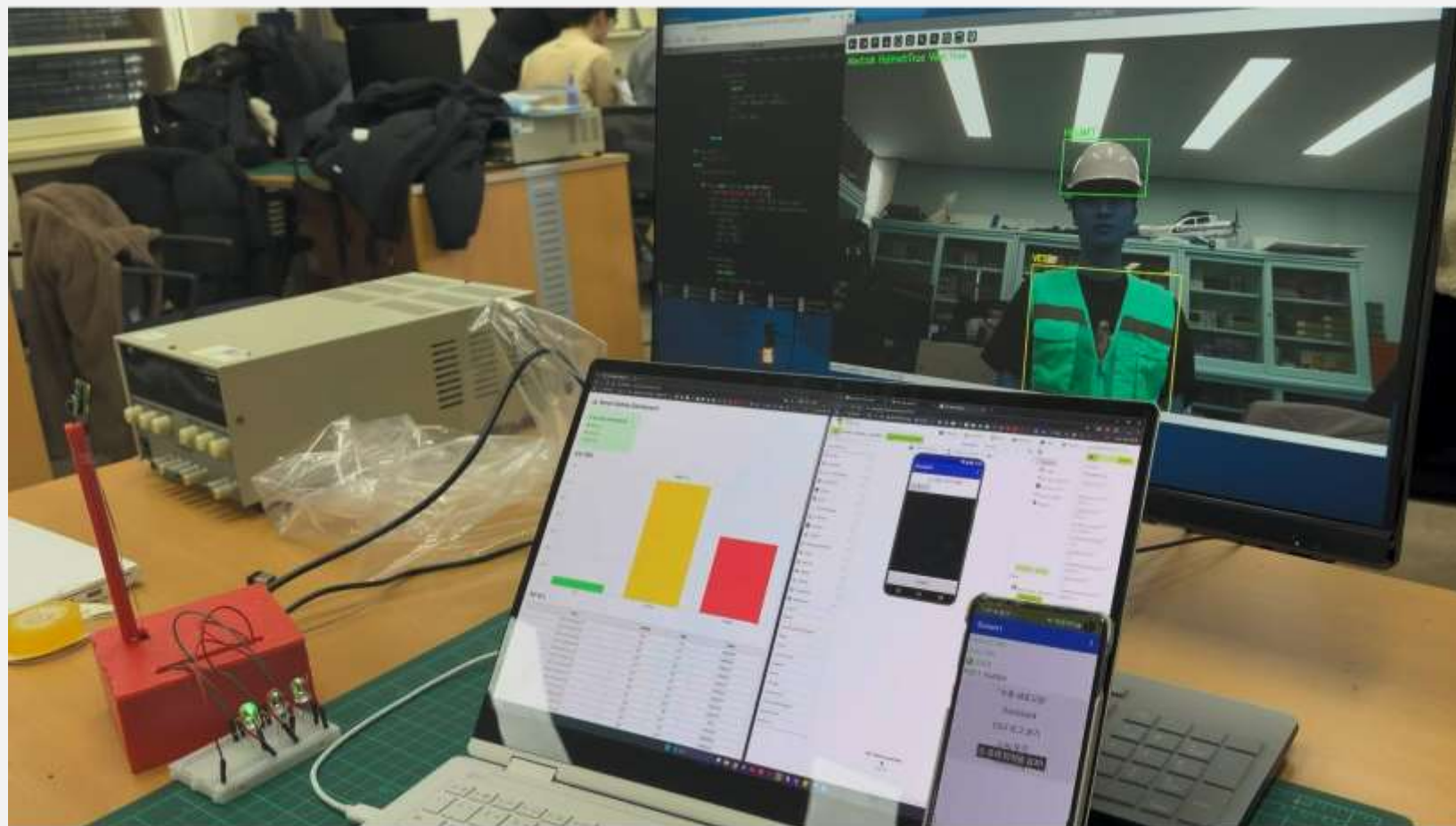
HELMET **ON**
VEST **OFF****WARNING**HELMET LED ●
VEST LED ●
STATUS LED ●

04.

HELMET **OFF**
VEST **OFF****DANGER**HELMET LED ●
VEST LED ●
STATUS LED ●

05 프로젝트 동작시연

프로젝트 작동 영상



06

현실적 제한요소 제한요소



객체 검출 로직의 한계 및 환경 의존성

- 조명 변화, 역광, 피사체 가림, 카메라 각도 등에 따라 탐지 성능에 차이가 발생
- Vest의 오인식 발생률이 높아지는 경향이 있었다



Device 성능 및 실시간 성능 확보의 제약

- 입력 해상도 증가 시 프레임 저하
- 다수 객체 등장 시 연산량 증가
- 탐지모델 변경 시 성능 편차 발생
- Hailo NPU사용을 고려하였으나 HEF 출력 포맷 부재로 인한 문제 발생



머리, 상체 매칭 규칙 복잡성

- YOLO학습을 계획하였으나 Vest의 복잡성으로 인해 학습 기간 등의 제약이 있음.



테스트 장비 부족

- 다양한 종류의 안전모, 안전조끼를 구하기 힘들



현실적 제한요소

캡스톤 디자인 06분반

Thank You

경청해 주셔서 감사합니다.

2022652 서주호
1922819 정세훈
2121944 김은영
2221301 정혁수
1823933 백시우